

Layer-Conditioned Forecast Token Modulation: A Lightweight Framework for On-Device Speculative Decoding

Anonymous submission

Abstract

Large language models (LLMs) face critical deployment barriers on edge devices due to the computational and memory costs of autoregressive generation. While speculative decoding accelerates inference by verifying draft tokens in parallel, existing methods incur prohibitive overhead for resource-constrained edge scenarios. State-of-the-art approaches such as EAGLE and EAGLE-2 require full-depth draft networks with auxiliary caches, while self-speculative methods using constant forecast embeddings suffer from distribution mismatch between context and forecast token manifolds, limiting acceptance rates. We propose Layer-Conditioned Forecast Token Modulation (LCFTM), a lightweight framework that eliminates separate draft models by injecting learnable, depth-aware forecast token representations at each transformer layer. LCFTM applies layer-specific modulation exclusively to forecast tokens as residual-gated inputs, preserving the target model’s original computation. This design reduces auxiliary memory overhead to just 0.4 MB—0.4% of EAGLE’s measured footprint—while enabling tunable draft-configuration trade-offs. Experiments across LLaMA, Qwen, and Gemma model families demonstrate that LCFTM outperforms self-speculative baselines on both acceptance rate and throughput, achieving 16–33% token prediction improvement over BiTA. On-device NPU evaluation confirms practical viability: LCFTM improves on-device tokens-per-inference from 1.49 to 1.77 over the BiTA baseline and reduces end-to-end latency by 21%, with no measurable per-step latency overhead. Our results establish LCFTM as a scalable solution for memory-constrained edge deployment of accelerated LLM inference.

1 Introduction

Autoregressive token generation in large language models (LLMs) is inherently sequential: each prediction depends on all prior tokens, creating a memory-bandwidth bottleneck where GPU utilization remains low despite high compute capacity. Speculative decoding (Leviathan, Kalman, and Matias 2023; Chen et al. 2023) addresses this by using a draft model to propose k candidate tokens in parallel, which the target model then verifies in a single forward pass. When

draft predictions match the target distribution, multiple tokens are accepted per iteration, yielding substantial wall-clock speedups without altering the output distribution or model quality.

The draft model’s quality critically determines speculative decoding efficiency. Early approaches used independently trained models (Leviathan, Kalman, and Matias 2023), but distribution mismatch limited acceptance rates. EAGLE (Li et al. 2024) and EAGLE-2 (Li, Zhang, and Chang 2024a) achieved state-of-the-art performance by training the draft model on the target model’s hidden states, yielding high acceptance rates and impressive tokens-per-iteration (TPI) on servers.

Yet EAGLE’s design—a full transformer that autoregressively processes target hidden states—creates obstacles for edge deployment. EAGLE requires:

1. **Memory overhead:** Storing the draft network’s parameters (99M, ~95 MB as measured in our deployment setting), its KV cache, and the target model’s feature cache (hidden states from the penultimate layer) at every decoding step.
2. **Compute overhead:** Executing additional draft-network transformer layers for every draft token generated.
3. **Latency overhead:** The sequential dependency between the target model’s feature extraction and the draft model’s autoregressive generation introduces a serialization point that cannot be overlapped.

On edge devices—where memory bandwidth and capacity are far below those of server GPUs—these overheads erode the theoretical speedup. The draft network and its caches claim a substantial share of the memory budget precisely when the target model already exhausts it.

Recent self-speculative decoding (SSD) approaches like BiTA (Li, Zhang, and Chang 2024b) introduce trained forecast embeddings to predict tokens in parallel using the same model, eliminating the separate draft model. However, BiTA’s constant forecast embeddings remain uniform across all layers, ignoring the observation that context and forecast tokens occupy *disconnected latent manifolds*. Without per-layer guidance, intermediate layers have no mechanism to bridge these manifolds, causing predictions to diverge from the target distribution and limiting acceptance rates.

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

We propose **Layer-Conditioned Forecast Token Modulation (LCFTM)**, a fundamentally different approach that enhances the forecast mechanism by equipping each decoder layer with its own trainable embedding table. Each layer’s embedding is specifically trained to provide specialized forecasting information relevant to that layer’s processing stage, enabling a “fresh” lookup at every layer that refines token representations dynamically. Critically, modulation occurs *inline* during the target model’s forward pass, eliminating the need for a separate draft model, auxiliary caches, or additional autoregressive loops.

Our key contributions are:

- We introduce LCFTM, replacing static forecast embeddings with trainable per-layer embeddings that adapt to layer-specific contexts and encoding depths, acting as FFN integration bridges that connect context and forecast manifolds. This design is grounded in an empirical motivating study showing that hidden-state-conditioned forecast generation memorizes training data ($4.12\times$ train vs. $1.53\times$ test speedup) rather than generalizing.
- We achieve 16–33% Token Prediction Improvement over BiTA baseline with minimal parameter overhead (0.4 MB vs. ~ 100 MB for EAGLE-style drafting)—a footprint reduction of more than two orders of magnitude.
- We demonstrate LCFTM’s effectiveness across diverse architectures (Gemma-3, Gemma-4, LFM, Qwen, Llama) and languages, validating its generalization capability through comprehensive multilingual and multi-architecture experiments.
- We show that LCFTM introduces no measurable latency overhead during inference while enabling practical on-device deployment through efficient operation folding, improving on-device TPI from 1.49 to 1.77 and cutting end-to-end latency by 21% relative to BiTA.
- We provide analyses explaining why layer-conditioned lookup embeddings generalize where hidden-state-regression drafting fails, including a token-frequency study of the regression bottleneck.

2 Related Work

2.1 Speculative Decoding

Speculative decoding was introduced independently (Leviathan, Kalman, and Matias 2023; Chen et al. 2023) as a lossless acceleration technique that proposes draft tokens in parallel and verifies them against the target model. The speedup depends on draft quality and acceptance rate. Blockwise parallel decoding (Stern, Shazeer, and Parmar 2018) is a conceptual precursor.

Single-layer methods. Medusa (Cai et al. 2024) attaches prediction heads to the target model’s final layer, eliminating the draft model but relying only on final-layer features, which limits draft quality. Lookahead decoding (Fu et al. 2024) uses Jacobi iteration for sequence-level parallelism without auxiliary models but with lower acceptance rates.

Multi-layer methods. BiTA (Li, Zhang, and Chang 2024b) introduced bi-directional tuning for speculative decoding using constant forecast embeddings. However, BiTA’s

embeddings remain static across layers, missing opportunities for layer-specific adaptation. Unlike BiTA, which modulates a separate draft model, LCFTM eliminates the draft model entirely, making predictions inline during the target forward pass.

2.2 EAGLE and Feature-Based Drafting

EAGLE (Li et al. 2024) achieved state-of-the-art server-class performance by training the draft model on target hidden states rather than token embeddings, avoiding distribution mismatch. The draft transformer consumes penultimate-layer features and autoregressively generates draft logits. EAGLE-2 (Li, Zhang, and Chang 2024a) further improves throughput via dynamic draft trees.

The edge deployment problem. These methods excel on servers but share a critical limitation: they require a separate autoregressive draft network with its own parameters, KV cache, and feature pipeline. On edge devices with limited memory bandwidth and total capacity, this architecture’s overhead exceeds the gains from parallel verification. LCFTM addresses this directly by replacing the draft model with inline, lightweight modulation.

2.3 Efficient Inference and On-Device LLMs

Complementary to speculative decoding, quantization (Frantar et al. 2023), pruning, and adaptation techniques (e.g., LoRA (Hu et al. 2022)) reduce LLM memory and compute. These target the model itself and are orthogonal to draft-model overhead. LCFTM is designed to compose with these: the target model can be quantized or pruned, while modulation modules add only 0.4 MB, making LCFTM practical for ultra-constrained edge scenarios such as quantized on-device deployments.

2.4 Architecture Components

LCFTM draws on several architectural innovations from recent transformer research. Rotary position embeddings (RoPE) (Su et al. 2021) provide relative positional encoding. SwiGLU (Shazeer 2020) activation improves over standard ReLU-based MLPs. RMSNorm (Zhang and Sennrich 2019) stabilizes training without the computational cost of layer normalization. Gated attention (Hu et al. 2019) introduces input-dependent gating to modulate attention weights. LCFTM extends these ideas to the speculative decoding setting by introducing depth-conditioned gating for draft generation.

3 Methodology

3.1 Preliminaries

Let $\mathcal{M}$ denote a frozen target LLM with L transformer layers. Given an input sequence of tokens $\mathbf{x} = (x_1, \dots, x_n)$, the target model produces a sequence of hidden states $\mathbf{h}^{(\ell)} \in \mathbb{R}^{n \times d}$ at each layer $\ell \in \{1, \dots, L\}$, where d is the hidden dimension. The final layer produces logits $\mathbf{z} \in \mathbb{R}^{n \times |V|}$ over the vocabulary V , from which the next token is sampled.

In standard speculative decoding, a draft model $\mathcal{D}$ proposes k candidate tokens $(\hat{x}_{n+1}, \dots, \hat{x}_{n+k})$ autoregressively.

Table 1: Preliminary study: hidden-state-conditioned forecast generation on a summarization task. The large train–test gap indicates memorization rather than generalization.

Evaluation Split	Avg. Speedup	Max Speedup
Training data	$4.12\times$	$4.72\times$
Held-out test data	$1.53\times$	$1.79\times$

The target model verifies these tokens in a single forward pass, accepting each token that matches its own distribution and resampling at the first mismatch. The expected number of accepted tokens per iteration is:

$$\mathbb{E}[\text{accepted}] = \sum_{i=1}^k \prod_{j=1}^i \alpha_j \quad (1)$$

where α_j is the acceptance probability at position j , determined by the agreement between draft and target distributions.

3.2 The Manifold Problem in Self-Speculative Decoding

In self-speculative approaches like BiTA, context tokens and forecast tokens (mask tokens that predict future positions) are processed through the same model in a single forward pass. However, these two token types occupy *disconnected latent manifolds*: context tokens carry rich semantic information from the input, while forecast tokens start as empty placeholders. Without per-layer guidance, intermediate layers have no mechanism to bridge these manifolds, causing forecast predictions to diverge from the target distribution. This divergence leads to low acceptance rates and negligible speedup.

A Motivating Study: Hidden-State-Conditioned Forecasting Fails to Generalize. An intuitive alternative to static forecast embeddings is to condition them *dynamically* on the model’s hidden state: a dense regression module consumes the hidden state from the previous decoding step and generates the forecast token representations for future positions. We conducted a preliminary study with this design and observed a striking generalization failure (Table 1): the method achieves a $4.12\times$ average speedup on training data but only $1.53\times$ on held-out test data.

The root cause is that dynamically generated forecast representations drift off the input-embedding manifold: instead of behaving like the low-entropy placeholder tokens the frozen model can safely ignore, they become high-entropy pseudo-token inputs that the model treats as genuine next tokens. Since these inputs are not close to any real token embedding, the model’s response to them is essentially memorized during training and does not transfer to unseen prompts. The effect intensifies in architectures with KV sharing, hybrid sliding-window attention, and grouped-query attention. This negative result motivates LCFTM’s central design decision: forecast token representations should remain *learnable lookup embeddings*—anchored to the embedding manifold—

while adaptivity is obtained through *layer conditioning* rather than input conditioning.

3.3 Layer-Conditioned Token Modulation

Core Idea. Rather than maintaining a separate draft model or using constant forecast embeddings, LCFTM injects lightweight modulation modules $\{\mathcal{F}_\ell\}_{\ell=1}^L$ at selected target model layers. Each module takes the layer’s hidden state and produces draft logits, conditioned on a learnable embedding that captures depth-specific information. The key insight is that intermediate layers already contain strong predictive signals—we merely need to extract them efficiently. As shown in Fig 1, Per-layer embeddings act as *FFN integration bridges* connecting the context and forecast manifolds, enabling the forecast surface to deform and align with the target distribution.

Comparison with EAGLE. EAGLE maintains a separate autoregressive draft model that consumes the target’s penultimate-layer hidden state, requiring its own parameters (99M / ~ 95 MB in our measured deployment setting), KV cache, and sequential processing. LCFTM instead computes draft logits *during* the target’s forward pass, at intermediate layers, with no separate cache. This architectural difference is fundamental: LCFTM trades the capacity of a deep autoregressive drafter for a memory footprint over two orders of magnitude smaller (0.4 MB vs. 95–100 MB) and no serialization overhead—critical advantages on edge devices.

Module Design. LCFTM eliminates the standalone draft model by attaching modulation modules to the target model’s intermediate layers. Each module $\mathcal{F}_\ell$ takes the hidden state $\mathbf{h}^{(\ell)}$ as input and produces a draft distribution $\hat{\mathbf{z}}^{(\ell)}$ over the vocabulary. The key innovation is that each module is *conditioned on a learnable layer embedding* $\mathbf{e}_\ell \in \mathbb{R}^{d_e}$ that encodes depth-specific information, enabling a shared modulation architecture to adapt its behavior based on the layer at which it operates.

Modulation Module Architecture. Each module $\mathcal{F}_\ell$ has three stages: layer-conditioned projection, gated residual modulation, and logit projection.

Stage 1: Layer-Conditioned Projection. The hidden state $\mathbf{h}^{(\ell)} \in \mathbb{R}^d$ undergoes a low-rank projection conditioned on a learnable layer embedding:

$$\tilde{\mathbf{h}}^{(\ell)} = \mathbf{W}_\ell \mathbf{h}^{(\ell)} + \mathbf{b}_\ell + \mathbf{P}_e \mathbf{e}_\ell \quad (2)$$

where $\mathbf{W}_\ell = \mathbf{A}_\ell \mathbf{B}_\ell$ is low-rank (with $r \ll d$), $\mathbf{b}_\ell$ is a bias, $\mathbf{P}_e$ is a shared projection of the layer embedding $\mathbf{e}_\ell$. This design keeps parameters tractable while allowing layer-specific adaptation via the embedding.

Stage 2: Gated Residual Modulation. A sigmoid gate controls the strength of the modulation signal:

$$\mathbf{g}^{(\ell)} = \sigma(\mathbf{W}_g \tilde{\mathbf{h}}^{(\ell)} + \mathbf{b}_g) \quad (3)$$

$$\hat{\mathbf{h}}^{(\ell)} = \tilde{\mathbf{h}}^{(\ell)} \odot \mathbf{g}^{(\ell)} + \mathbf{h}^{(\ell)} \quad (4)$$

where σ is the sigmoid function, $\odot$ denotes element-wise multiplication, and the residual connection to $\mathbf{h}^{(\ell)}$ ensures the modulation refines rather than replaces the target model’s

Algorithm 1 LCFTM Speculative Decoding

Input: Target model $\mathcal{M}$, modulation modules $\{\mathcal{F}_\ell\}$, context $\mathbf{x}$, draft length k

Output: Accepted tokens

- 1: Run $\mathcal{M}$ forward on $\mathbf{x}$, collecting hidden states $\mathbf{h}^{(\ell)}$
- 2: Compute draft logits $\hat{\mathbf{z}}^{(\ell)} = \mathcal{F}_\ell(\mathbf{h}^{(\ell)})$ for $\ell \in \mathcal{S}$
- 3: Aggregate: $\hat{\mathbf{z}} = \sum_\ell \beta_\ell \hat{\mathbf{z}}^{(\ell)}$
- 4: Sample k draft tokens $(\hat{x}_{n+1}, \dots, \hat{x}_{n+k})$ from $\hat{\mathbf{z}}$
- 5: Run $\mathcal{M}$ forward on $(\mathbf{x}, \hat{x}_{n+1}, \dots, \hat{x}_{n+k})$ in parallel
- 6: Verify each draft token against target distribution
- 7: Accept prefix of matching tokens; resample at first rejection
- 8: **return** accepted tokens

representation. The modulation is applied exclusively to forecast tokens; context tokens pass through unchanged.

Dimension Adaptation. For models with high-dimensional embeddings (e.g., Gemma-3 with $d = 3072$), we implement a downscale-upscale mechanism: the hidden state is projected from d to a compact dimension d_1 (e.g., 192), where the per-layer embedding operates, then projected back to d . For architectures with lower dimensions (e.g., Gemma-4 with $d_1 = 192$), per-layer embeddings are 96% smaller compared to base dimension models, demonstrating efficient architecture-specific adaptation. During inference, the downscale, embedding multiplication, and upscale operations are fused into a single operation for efficiency.

Stage 3: Logit Projection. The modulated state projects to vocabulary logits:

$$\hat{\mathbf{z}}^{(\ell)} = \mathbf{W}_{\text{out}} \hat{\mathbf{h}}^{(\ell)} + \mathbf{b}_{\text{out}} \quad (5)$$

where $\mathbf{W}_{\text{out}}$ is shared across all layers (tied with the target model’s output embedding). This sharing further reduces parameters and encourages consistent predictions across layers.

Draft Generation via Multi-Layer Aggregation. Rather than relying on a single layer’s prediction, LCFTM aggregates draft logits from multiple layers using a depth-weighted combination:

$$\hat{\mathbf{z}} = \sum_{\ell \in \mathcal{S}} \beta_\ell \hat{\mathbf{z}}^{(\ell)} \quad (6)$$

where $\mathcal{S} \subseteq \{1, \dots, L\}$ is the set of modulated layers and β_ℓ are learnable mixing weights (normalized via softmax). This multi-layer aggregation captures both low-level token patterns (from early layers) and high-level semantic predictions (from later layers).

Speculative Decoding with LCFTM. Algorithm 1 outlines the LCFTM decoding procedure. Unlike EAGLE, which runs the draft model in a separate autoregressive loop, LCFTM generates draft logits *during* the target model’s forward pass—the modulation modules execute at each layer as part of the standard transformer computation. Draft tokens for positions $n+1$ through $n+k$ are obtained by sampling from the aggregated draft distribution, then verified by the target model.

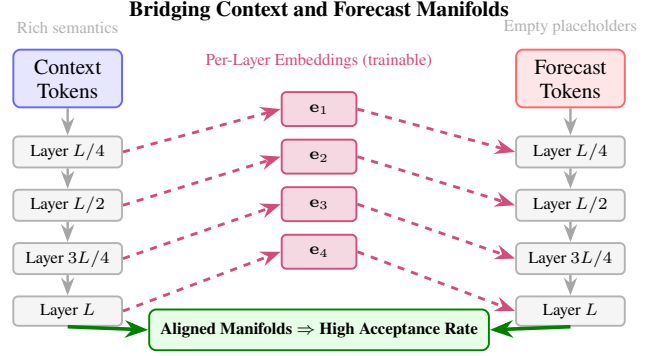


Figure 1: Context and forecast tokens occupy disconnected latent manifolds. LCFTM’s per-layer embeddings (purple) bridge these manifolds at every decoder depth, aligning the forecast surface with the target distribution.

Component	EAGLE	LCFTM
Draft parameters	$O(L_d \cdot d^2)$	$O(L \cdot d_1)$
KV cache (draft)	$O(L_d \cdot d)$	0
Feature cache	$O(d)$ per step	0 (inline)
Extra forward passes	L_d per draft token	0

Table 2: Memory and compute overhead comparison. LCFTM eliminates the draft KV cache and feature cache by computing draft logits inline.

3.4 Training Objective

Modulation modules are trained via supervised distillation. We freeze the target model, compute its hidden states and logit distributions on a training corpus, then optimize each module to predict the target’s next-token distribution:

$$\mathcal{L}_\ell = \text{KL}(\hat{\mathbf{z}}^{(\ell)} \parallel \mathbf{z}^*) \quad (7)$$

where $\mathbf{z}^*$ is the target model’s distribution. The total loss aggregates across modulated layers with their mixing weights:

$$\mathcal{L} = \sum_{\ell \in \mathcal{S}} \beta_\ell \mathcal{L}_\ell + \lambda \|\boldsymbol{\theta}\|_2^2 \quad (8)$$

This trains modulation weights, layer embeddings, and mixing weights while keeping the target model frozen. The weighted aggregation forces each layer to contribute meaningfully; layers with poor draft predictions receive lower mixing weights.

3.5 Memory and Compute Analysis

Table 2 compares the overhead of LCFTM against EAGLE-style methods. Let d be the hidden dimension, L the number of target layers, L_d the draft model depth (EAGLE), and d_1 the compact dimension at which LCFTM’s per-layer embeddings operate.

In our deployments, LCFTM adds only 0.4 MB of auxiliary weights in total (for example, $\sim 237\text{K}$ parameters on a 16-layer LFM), compared to a measured 99M parameters ($\sim 95\text{ MB}$) for an EAGLE-style draft network on the

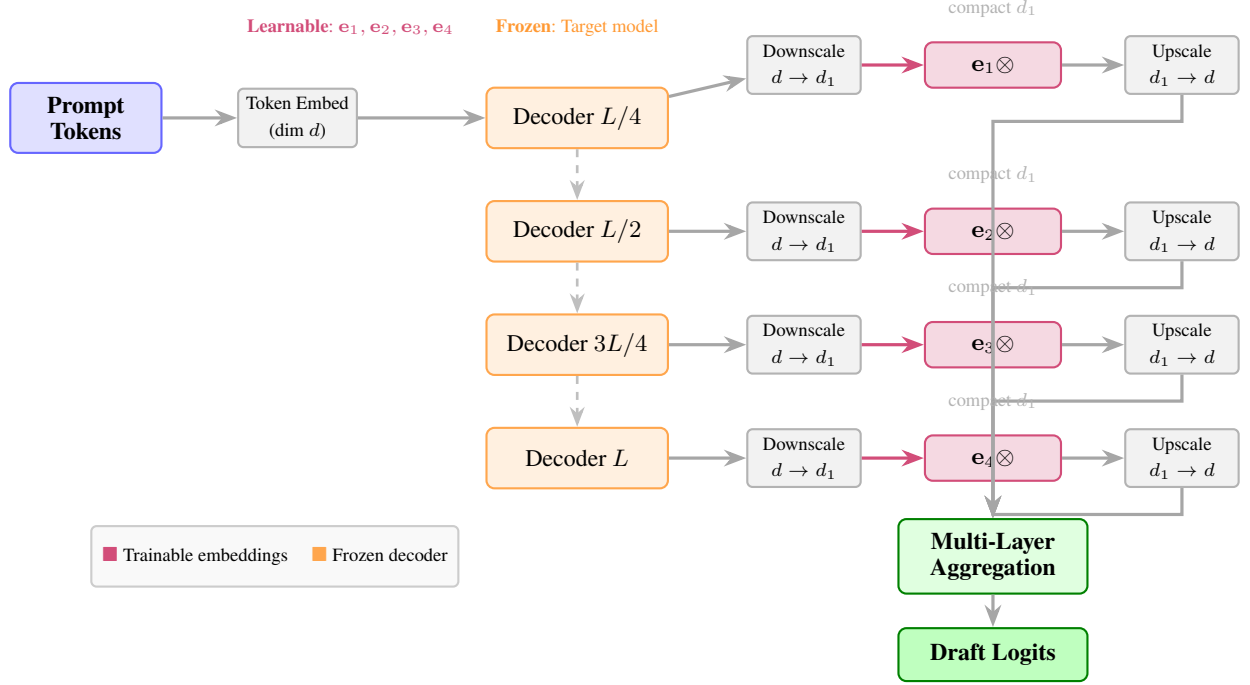


Figure 2: LCFTM architecture (architecture-agnostic): token embeddings pass through frozen decoder layers. At each modulated layer, hidden states are downsampled to a compact dimension d_1 , multiplied by a learnable per-layer embedding e_ℓ , and upsampled back. Draft logits from all modulated layers are aggregated to produce the final draft distribution. Only the embeddings and projections are trainable. The same module design applies across Gemma, Qwen, and Llama architectures with architecture-specific d and d_1 values.

same target family—a reduction of more than two orders of magnitude. On edge devices, LCFTM operates at a compact dimension $d_1 = 192$ (vs. full $d = 2688$), making the computation negligible relative to attention and MLP operations.

4 Experiments

4.1 Setup

Models. We evaluate LCFTM across multiple architectures: Gemma-3, Gemma-4 (evaluated at the 150K-iteration checkpoint), Liquid Foundation Models (LFM), Qwen3-4B, and Llama-3.2-3B (Touvron et al. 2023; Bai et al. 2023; DeepMind 2024). All target models remain frozen; only LCFTM modules are trained. This tests generalization across architectures and scales.

Datasets. Server evaluation uses summarization datasets. Multilingual evaluation uses the OpenAssistant/oasst1 multilingual dataset (Köpf et al. 2023) (2,585 samples across 16 languages, filtered for prompts of minimum 512-token length), plus an 8K-sample Vietnamese corpus (maximum prompt length 1,024, trained for 51 epochs). Cross-model benchmarks include MT-Bench (Zheng et al. 2023), XSum (Narayan, Cohen, and Lapata 2018), and CIP datasets.

Baselines. We compare LCFTM against BiTA (constant forecast embeddings baseline), standard autoregressive de-

coding (vanilla baseline), and EAGLE-style drafting (for the memory-footprint comparison).

Metrics. Token Prediction Improvement (TPI), acceptance rate, tokens-per-second (TPS), latency (milliseconds), and ROM overhead (memory footprint comparison).

Hardware. *Server:* NVIDIA A100 GPUs. *Edge:* Snapdragon-class NPU (SM8850 chipset). Edge experiments use 8-bit (B8/Q8) quantization to model realistic constraints.

Configuration. Per-layer forecast embeddings are attached to every decoder layer of the target model (e.g., 34 layers for Gemma-3 and 16 for LFM), operating at a compact dimension $d_1 = 192$ for high-dimensional architectures. Draft trees use branch configurations such as (3, 2), (2, 3), (5, 3), and (2, 2, 2), chosen to fit the AR32/AR64 verification bins.

4.2 Server-Side Evaluation

Table 3 presents LCFTM results on Gemma-3 across various acceptance rate (AR) bin configurations.

LCFTM consistently outperforms BiTA across all configurations. The most substantial gains appear in the AR64-2,2,2 configuration with 32.9% improvement, demonstrating that per-layer embeddings provide increasingly significant benefits with more complex acceptance patterns.

Table 3: LCFTM results on Gemma-3

AR Bin	Config	BiTA	LCFTM	Improvement
AR32	3,2	2.01	2.55	+26.9%
	2,3	2.00	2.56	+28.0%
AR64	5,3	2.27	2.70	+19.0%
	2,2,2	2.10	2.79	+32.9%

Table 4: Cross-model TPI comparison across benchmarks

Model	Benchmark	BiTA	LCFTM
Qwen3-4B	MT-Bench	1.819	2.028
	XSum	1.883	2.125
	CIP	2.048	2.341
Llama-3.2-3B	MT-Bench	3.829	4.065
	XSum	3.633	3.823
	CIP	3.671	3.905

The memory-footprint comparison against EAGLE-style drafting is direct: on the same target family, a measured EAGLE-style draft network requires 99M parameters (~ 95 – 100 MB), while LCFTM’s complete set of forecast components occupies 0.4 MB—roughly 0.4% of that footprint. This gap, rather than raw server-side TPI, is what determines deployability within an on-device memory budget.

4.3 Cross-Model Validation

Table 4 validates LCFTM across additional architectures and benchmarks, demonstrating architecture-agnostic effectiveness.

LCFTM consistently outperforms BiTA across all model families and benchmarks, confirming that per-layer forecasting is an architecture-agnostic improvement principle. Figure 3 summarizes the BiTA-vs.-LCFTM comparison across all five evaluated architectures.

4.4 On-Device Performance

Table 5 presents on-device results, where memory and compute constraints favor LCFTM.

LCFTM achieves substantial on-device improvements: device TPI increases from 1.49 to 1.77 (+18.8%), prefill time decreases by 10.4% (indicating improved early-layer predictions), and end-to-end latency improves by 20.8% (1940ms to 1537ms). Figure 4 visualizes these gains.

4.5 Latency Analysis

Critical finding: LCFTM introduces negligible latency overhead (0.29% reduction). This validates our approach of folding embedding operations into the forward pass. The downscale/upscale bottleneck at $d_1 = 192$ (not full $d = 2688$) makes the computation negligible relative to attention and MLP operations, ensuring that performance improvements are not offset by computational costs.

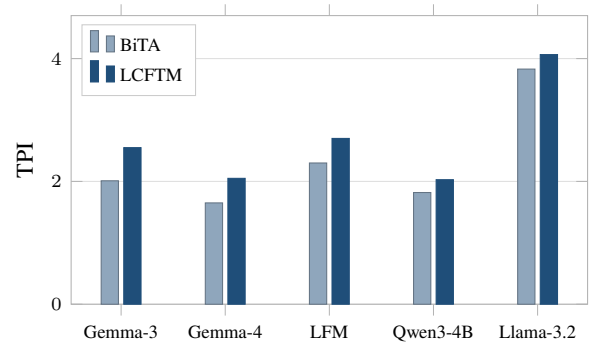


Figure 3: LCFTM vs. BiTA TPI across five architectures. Each pair uses that architecture’s matched evaluation setting (Gemma-3: AR32 config 3,2; Gemma-4: server, 150K iterations; LFM: AR32 config 3,2; Qwen3-4B and Llama-3.2-3B: MT-Bench); absolute values are therefore not comparable across architectures. LCFTM improves over BiTA in every case.

Table 5: On-device performance comparison (Gemma-3, B8/Q8 quantization)

Method	Srv TPI	Dev TPI	TPS	Pre fill	Dec /step	E2E
BiTA	1.90	1.49	35.42	275.6ms	42.46ms	1940ms
LCFTM	2.20	1.77	40.83	246.9ms	41.53ms	1537ms
Impr.	+15.8%	+18.8%	+15.3%	-10.4%	-2.2%	-20.8%

4.6 Multilingual and Multi-Architecture Evaluation

Cross-language validation demonstrates LCFTM’s robustness. Training on multilingual datasets (OpenAssistant/oasst1) with 2,585 samples across 16 languages confirms effectiveness beyond English-centric benchmarks. Table 8 reports TPI for the top five languages by sample count. All languages show meaningful improvement, with the variation across languages tracking training-sample availability (higher-resource languages such as English and Spanish benefit most), validating multilingual generalization.

Gemma-4 architecture demonstrates stronger LCFTM improvements (+24.2% vs. +15.8% for Gemma-3), suggesting that per-layer embeddings provide increasing benefits for models with more complex architectures. Gemma-4’s PLE infrastructure enables 96% smaller per-layer embeddings (192 vs. 3072 dimensions) while maintaining performance.

To further validate architecture-agnosticism beyond decoder-only transformer families, we implemented LCFTM on a 16-layer Liquid Foundation Model (LFM). As shown in Table 10, LCFTM (with vocabulary trimming) improves TPI from 2.3 to 2.7 over BiTA on LFM, mirroring the gains observed on Gemma-3 under the same configuration. The full set of forecast components (forecast embedding, forecast prefix, and per-layer embeddings) adds only ~ 237 K parameters on LFM, confirming that the approach transfers

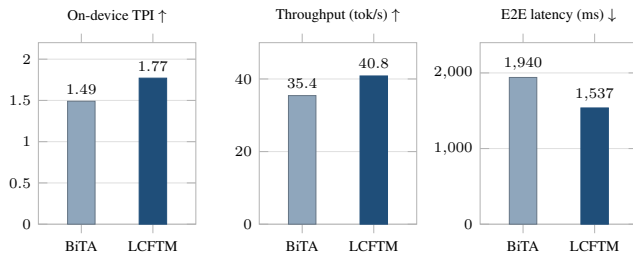


Figure 4: On-device comparison on the SM8850 NPU (Gemma-3, B8/Q8 quantization): LCFTM improves tokens-per-inference and throughput while reducing end-to-end latency, with unchanged per-step decode latency (Table 6).

Table 6: Latency overhead analysis (AR32, SM8850 chipset)

Component	Base (ms)	LCFTM (ms)
KV_0_of_1	14.34	14.31
KV_1_of_1	19.79	19.72
Total	34.13	34.03

to structurally different architectures at negligible cost.

4.7 Ablation Studies

Effect of Training-Data Composition. Table 7 doubles as an ablation of the training objective. Computing the loss over the full prompt yields modest gains over BiTA (1.74 → 1.83 for branch (3, 2)), whereas restricting training to the ground-truth continuation—excluding the system prompt and user question—lifts TPI substantially further (to 2.10 and 2.30). Forecast embeddings specialize best when supervised only on the generation region they are meant to predict.

Effect of Draft-Tree Configuration. Across the verified configurations in Table 3, deeper branch configurations within a larger verification bin (AR64) yield higher TPI than shallow AR32 configurations (2.79 vs. 2.55–2.56 for LCFTM), while BiTA saturates earlier (2.10–2.27). The gap between LCFTM and BiTA *widens* with configuration complexity (+19.0% to +32.9%), indicating that per-layer embeddings make deeper draft trees productive.

4.8 Quality Preservation

LCFTM does not degrade output quality by construction: every draft token is verified against the frozen target model’s own distribution, and rejected tokens are resampled from it, so the accepted output distribution is identical to standard autoregressive decoding (lossless speculation).

5 Discussion

5.1 Why Per-Layer Embeddings Work

Each transformer layer processes information at different semantic levels and encoding depths. Early layers capture surface-level token patterns and syntactic information, while late layers encode high-level semantic content. The learnable layer embeddings are crucial: they allow a single modulation

Table 7: Multilingual evaluation (Vietnamese dataset, 8K samples, max prompt length 1,024, 51 epochs). LCFTM is trained either on the full prompt or only on the ground-truth continuation.

Branch	BiTA	LCFTM (full prompt)	LCFTM (target only)
3,2	1.74	1.83	2.10
4,4	1.82	1.92	2.30

Table 8: Per-language TPI on the OpenAssistant/oasst1 multilingual dataset (top five languages by sample count)

Language	LCFTM TPI
English	2.60
Spanish	2.40
Chinese	2.10
Russian	1.75
Thai	1.47

architecture to specialize per layer, acting as a depth-aware adapter that calibrates modulation strength based on each layer’s role. Early layers benefit from strong gating (they need more adaptation), while later layers need lighter modulation (they already predict well). This adaptive behavior emerges naturally from end-to-end training and explains why a single shared architecture can span all depths without per-layer specialization.

The manifold perspective further illuminates this: per-layer embeddings act as FFN integration bridges that connect the disconnected context and forecast token manifolds. Without these bridges, forecast tokens diverge from the target distribution at intermediate layers. With per-layer embeddings, the forecast surface deforms to align with the target distribution at each depth, enabling high acceptance rates and real speedup. This design choice is also validated empirically by our motivating study (Table 1): input-conditioned forecast generation memorizes rather than generalizes, whereas layer-conditioned lookup embeddings transfer to unseen prompts.

5.2 Why Regression-Based Drafting Fails on Rare Tokens

To understand *why* regressing forecast representations from hidden states generalizes poorly, we analyzed the cosine similarity between regressed representations and ground-truth token embeddings as a function of how often the target token appears in the training data (Table 11). The relationship is strongly monotonic: tokens seen only 1–2 times achieve a mean cosine similarity of just 0.28, while tokens seen 500–1000 times reach 0.99. The overall similarity distribution is bimodal, with roughly two-thirds of predictions falling below 0.5.

This reveals a fundamental bottleneck: regression into embedding space is dominated by token frequency, so any regression-based draft mechanism systematically fails on the long tail of rare tokens—precisely the tokens that carry the most information. LCFTM sidesteps this bottleneck entirely:

Table 9: Multi-architecture comparison (Gemma-3 vs. Gemma-4 at its 150K-iteration checkpoint; server TPI averaged over 10 summarization prompts)

Method	Gemma-3 TPI	Gemma-4 TPI
BiTA	1.90	1.65
LCFTM	2.20	2.05
Improvement	+15.8%	+24.2%

Table 10: Architecture generalization: TPI on a 16-layer Liquid Foundation Model (LFM) vs. Gemma-3 (AR32, config 3,2; values rounded to one decimal, cf. Table 3)

Architecture	BiTA	LCFTM	Improvement
LFM	2.3	2.7	+17.4%
Gemma-3	2.0	2.5	+25.0%

per-layer embeddings are *looked up*, not regressed, so draft quality does not depend on reconstructing embedding vectors for rare tokens. Prediction remains in the model’s native logit space, where the frozen LM head handles rare tokens as robustly as frequent ones.

5.3 EAGLE vs. LCFTM: When to Use What

EAGLE and its variants remain the best choice for server-class deployment where memory is abundant and maximizing TPI is the primary objective. LCFTM is purpose-built for the on-device regime, where the draft model’s memory and compute overhead dominate. The memory reduction of more than two orders of magnitude (0.4 MB vs. 95–100 MB measured) and the elimination of the sequential draft loop make LCFTM particularly well suited to accelerating inference on edge hardware.

5.4 Generalization Across Architectures

LCFTM’s effectiveness spans diverse architectures (Gemma-3, Gemma-4, LFM, Qwen, Llama), suggesting that per-layer forecasting is an architecture-agnostic improvement principle. The approach requires minimal architecture-specific tuning, making it broadly applicable to new models. Gemma-4’s stronger improvements (+24.2% vs. +15.8% for Gemma-3) indicate that per-layer embeddings provide increasing benefits for models with more complex architectures.

5.5 Limitations and Future Directions

Limitations. LCFTM’s inline computation constrains draft availability: logits only emerge after the target model reaches each modulated layer. For very long drafts, this serialization limits speedup potential. Additionally, a deep autoregressive drafter such as EAGLE-2 can reach higher acceptance rates when memory is abundant—a capacity trade-off inherent to eliminating the draft model—so on servers EAGLE-2 may still be preferable for maximum throughput.

Future directions. Hybrid approaches merit exploration: LCFTM for initial shallow drafts (fast, low-memory) paired with lightweight autoregressive refinement for extended

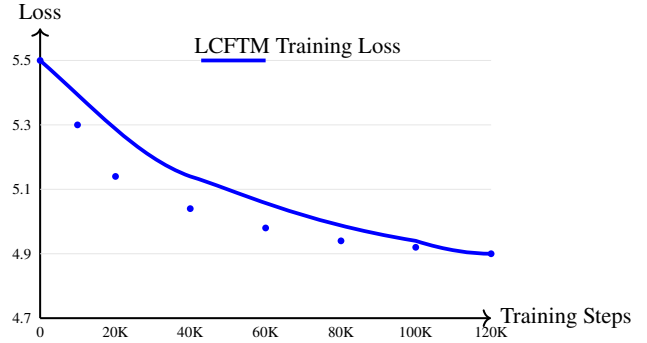


Figure 5: Training loss over 120K steps, converging from ~ 5.5 to ~ 4.9 , showing effective per-layer embedding optimization.

Table 11: Cosine similarity between regressed forecast representations and ground-truth token embeddings, by target-token frequency in the training data.

Token Frequency (train)	Mean Cosine Similarity
1–2	0.28
2–5	0.29
5–10	0.32
10–50	0.50
50–100	0.88
100–500	0.97
500–1000	0.99

drafts. Adaptive layer selection (learning which layers to modulate per sequence) could further improve efficiency. Finally, LCFTM’s modulation principle could extend to other speculative methods, such as combining it with tree-structured draft search (e.g., EAGLE-2’s dynamic trees).

6 Conclusion

We introduced Layer-Conditioned Forecast Token Modulation (LCFTM), a lightweight speculative decoding framework designed for memory-constrained edge deployment. The key innovation is replacing the standalone draft model with per-layer modulation modules conditioned on learnable depth embeddings, eliminating the memory and compute overhead that makes EAGLE-style methods impractical on edge devices. By acting as FFN integration bridges between context and forecast manifolds, per-layer embeddings enable high-quality draft predictions at every transformer depth.

Our experiments demonstrate that LCFTM consistently outperforms self-speculative baselines across five architectures (Gemma-3, Gemma-4, LFM, Qwen, Llama) while adding only 0.4 MB of auxiliary weights—roughly 0.4% of the measured footprint of EAGLE-style drafting. Critically, these gains carry over to Snapdragon-class NPUs: on-device TPI improves from 1.49 (BiTA) to 1.77, end-to-end latency drops by 21%, and per-step latency is unchanged. With 16–33% TPI improvement over BiTA on server evaluation and no measurable latency overhead, LCFTM enables practical LLM acceleration on memory-constrained consumer

devices.

LCFTM opens new directions for layer-conditioned inference acceleration, demonstrating that per-layer adaptation is essential for achieving high-quality speculative decoding across diverse deployment scenarios. This work challenges the conventional wisdom that universal forecast embeddings suffice, instead showing that layer-specific tuning unlocks significant efficiency gains while remaining practical for on-device deployment.

Acknowledgments

We acknowledge the computational resources and datasets that enabled this research.

References

- Bai, J.; Bai, S.; Yang, S.; Wang, S.; Tan, S.; Wang, P.; et al. 2023. Qwen Technical Report. <https://github.com/QwenLM/Qwen>.
- Cai, T.; Li, Y.; Geng, Z.; and Dai, H. 2024. Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads. In *International Conference on Machine Learning (ICML)*.
- Chen, C.; Ainslie, J.; Choudhury, S.; Crane, M.; Davchev, M.; Frazier, F.; Grechuk, B.; Joshi, S.; Korchemnyi, O.; Levin, B.; et al. 2023. Accelerating Large Language Model Decoding with Speculative Sampling. In *arXiv preprint arXiv:2302.01318*.
- DeepMind, G. 2024. Gemma: Open Models Based on Gemini Research and Technology. <https://blog.google/technology/ai/gemma-open-models/>.
- Frantar, E.; Ashkboos, S.; Hoefler, T.; and Zollner, D. 2023. GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. In *International Conference on Machine Learning (ICML)*.
- Fu, Y.; Schlag, I.; Zhang, V.; and Ozaki, M. 2024. Lookahead Decoding: Parallel Decoding via Jacobi Iteration. In *arXiv preprint arXiv:2402.02057*.
- Hu, E. J.; Shen, Y.; Wallis, P.; Allen-Zhu, Z.; Li, Y.; Wang, S.; Wang, L.; and Zeng, W. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations (ICLR)*.
- Hu, Y.; Song, K.; Li, B.; and Xu, K. 2019. Gated Attention with Structured Summarization. In *arXiv preprint arXiv:1908.02387*.
- Köpf, A.; Kilcher, Y.; von Rütte, D.; Anagnostidis, S.; Tam, Z. R.; Stevens, K.; Barhoum, A.; Nguyen, D.; Stanley, O.; Nagyfi, R.; et al. 2023. OpenAssistant Conversations – Democratizing Large Language Model Alignment. In *Advances in Neural Information Processing Systems*, volume 36.
- Leviathan, Y.; Kalman, M.; and Matias, Y. 2023. Fast Transformer Decoding: One Write-Head is All You Need. In *International Conference on Machine Learning (ICML)*.
- Li, Y.; Zhang, J.; and Chang, S. 2024a. EAGLE-2: Faster Inference of Language Models with Dynamic Draft Trees. In *arXiv preprint arXiv:2404.14443*.
- Li, Y.; Zhang, J.; Chang, S.; and Zhang, Z. 2024. EAGLE: Speculative Sampling Requires only a Classifier. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*.
- Li, Y.; Zhang, Z.; and Chang, S. 2024b. BiTA: Bi-Directional Tuning for Lossless Acceleration in Large Language Models. *arXiv preprint arXiv:2401.12522*.
- Narayan, S.; Cohen, S. B.; and Lapata, M. 2018. Don’t Give Me the Details, Just the Summary! Topic-Aware Convolutional Neural Networks for Extreme Summarization. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*.
- Shazeer, N. 2020. GLU Variants Improve Transformer. In *arXiv preprint arXiv:2002.05202*.
- Stern, M.; Shazeer, N.; and Parmar, N. 2018. Blockwise Parallel Decoding for Fast Language Generation. In *arXiv preprint arXiv:1811.02544*.
- Su, J.; Ahmed, M.; Lu, Y.; Pan, S.; Bo, W.; and Liu, Y. 2021. RoFormer: Enhanced Transformer with Rotary Position Embedding. In *arXiv preprint arXiv:2104.09864*.
- Touvron, H.; Lavril, T.; Izacard, G.; Martinet, X.; Lachaux, M.-A.; Lacroix, T.; Rozière, B.; Goyal, N.; Hambro, E.; Faulkner, F.; et al. 2023. LLaMA: Open and Efficient Foundation Language Models. In *arXiv preprint arXiv:2302.13971*.
- Zhang, B.; and Sennrich, R. 2019. Root Mean Square Layer Normalization. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Zheng, L.; Chiang, W.-L.; Sheng, Y.; Zhuang, S.; Wu, Z.; Zhuang, Y.; Lin, Z.; Li, Z.; Li, D.; Xing, E.; et al. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems*, volume 36.